

CHAPTER THREE

SYSTEM ANALYSIS, DESIGN METHODOLOGY, TOOLS AND IMPLEMENTATION APPROACH

Project Title: GradAssess AI

An AI-Powered Graduate Employability Assessment Platform

3.0 Introduction

This chapter presents the methodology, tools, architectural decisions, implementation strategy, and system design adopted in the development of **GradAssess AI**, an AI-powered graduate employability assessment platform. The system was developed to solve the major limitations observed in traditional graduate assessment processes, such as slow grading, weak integrity monitoring, poor student feedback cycles, and limited administrative control over assessment operations.

The platform combines a modern web-based user interface with a structured backend, persistent database storage, automated assessment grading, AI-assisted practice mode, certificate generation, results analysis, and career insight features. In addition to providing a student-facing examination environment, the system also includes an administrator workspace for managing users, assessments, attempts, certificates, and security incidents.

The purpose of this chapter is therefore to explain how the system was conceived and implemented from a technical point of view. It covers the development methodology used, software and services employed, system architecture, database structure, functional modules, security design, AI integration strategy, and the implementation workflow used in building the application.

3.1 Development Methodology

3.1.1 Adopted Methodology

The methodology adopted for this project is an **Agile incremental prototyping methodology**. This approach was selected because the platform contains multiple interdependent modules, including authentication, dashboard views, assessment delivery, automatic grading, AI-assisted practice, certificate management, results analysis, career insight generation, and administrative operations. Building such a system in one rigid cycle would have made iteration difficult, especially where interface, logic, and user workflow had to evolve together.

Under the Agile prototype approach, the system was implemented in small functional units. Each unit was developed, reviewed, refined, and extended before the next dependent unit was built. For example, the authentication workflow was established before protected student and admin dashboards were linked to live data. Likewise, the assessment-taking interface was built before proctoring event logging, auto-grading, and certificate issuance were fully connected.

3.1.2 Why Agile Was Suitable for This Project

The Agile approach was suitable for the following reasons:

1. **It supported continuous refinement of user interfaces.** The student and admin dashboards required repeated design corrections to improve usability, clarity, and responsiveness.
2. **It made backend integration easier.** Features could first be built with mock data and later replaced with live API integration without discarding the interface.
3. **It accommodated evolving requirements.** As the project grew, features such as practice review, AI-assisted grading, live admin security alerts, and dynamic career insight generation were added or refined based on observed needs.
4. **It reduced risk.** Individual modules could be tested independently before being merged into the main system workflow.

3.1.3 Major Development Phases

The implementation process followed the stages below:

1. Requirement identification and feature planning.
2. User interface prototyping with reusable frontend components.
3. Backend API and database schema development.
4. Integration of frontend and backend services.
5. Integration of AI-based generation and grading support.
6. Testing, correction, refinement, and usability adjustments.

3.2 Requirement Analysis

3.2.1 Functional Requirements

The developed system satisfies a number of major functional requirements. For the student side, the platform enables users to register and log in securely, access a personal dashboard, view assessments, take real assessments in fullscreen mode, practice with AI-generated questions, review performance, download certificates, and receive career insight recommendations. For the administrative side, the system allows administrators to create and edit assessments, manage students, monitor attempts, review security alerts, manage certificates, and configure application settings.

The assessment engine supports three question types, namely multiple choice questions (MCQ), theory/long answer questions, and coding questions. The system also supports automatic grading after submission, certificate issuance for passed attempts, tracking of suspicious assessment behaviour, and live review of flagged incidents from the administrator dashboard.

3.2.2 Non-Functional Requirements

The important non-functional requirements considered include:

- Responsiveness across desktop, tablet, and mobile devices.
- Security of authentication and role-based access control.
- Reliability of assessment session handling and result persistence.
- Scalability through modular backend and reusable frontend components.
- Usability through clean and focused user interface design.
- Maintainability through clear module boundaries and typed APIs.

3.3 System Architecture

GradAssess AI adopts a **client-server architecture**. The frontend client handles user interaction, state presentation, and browser-level proctoring support, whereas the backend provides authentication, business logic, persistence, grading, certificate management, and administrative reporting. The database layer stores users, assessments, questions, attempts, certificates, settings, sessions, and AI artifacts.

Architecture Summary: The system consists of a React/Next.js frontend, a NestJS backend API, a PostgreSQL database managed through Prisma ORM, and Gemini-based AI services for practice generation, narrative support, and open-ended grading assistance.

3.3.1 Frontend Layer

The frontend was built with **Next.js using the App Router, React 19, Tailwind CSS, and Framer Motion**. This layer provides the entire visual interaction model of the system. It includes the student dashboard, assessment catalogue, instructions page, live assessment-taking interface, practice mode, results page, certificate pages, career insights, settings, and all admin dashboards.

The frontend is highly componentized. Reusable components were created for cards, navigation, metric panels, status badges, modals, forms, question renderers, palettes, action menus, security

drawers, and certificate previews. This component-based approach simplified maintenance and allowed consistent styling across the product.

3.3.2 Backend Layer

The backend was implemented with **NestJS**, which provides a modular and scalable architecture for APIs. The system backend is separated into functional modules, including:

- **Auth Module** for registration, login, token refresh, logout, forgot password, reset password, and profile retrieval.
- **Exams Module** for assessment creation, assessment listing, exam session start, submission, grading, results, and career insights.
- **Admin Module** for overview analytics, user management, assessment management, attempts monitoring, certificate operations, and security review.
- **AI Module** for practice generation, practice review, AI-assisted grading, and narrative generation.
- **Certificates Module** for issuing, verifying, listing, and retrieving certificates.
- **Settings Module** for profile updates, preferences, notification settings, privacy controls, and session/device management.
- **Proctoring Module** for receiving and storing assessment integrity events.

3.3.3 Database Layer

The database layer uses **PostgreSQL** as the primary relational database and **Prisma ORM** as the data-access layer. PostgreSQL was selected because it is robust, widely adopted, and suitable for transactional systems that require data integrity. Prisma was adopted because it provides a strongly typed schema, typed queries, migrations, and improved developer productivity.

3.4 System Design

3.4.1 Major System Actors

The system was designed around two principal actors:

1. **Student:** The student can create an account, log in, configure profile settings, view assessments, take assessments, practise with AI-generated questions, view results, download certificates, and access career insights.
2. **Administrator:** The administrator can monitor the platform, manage students, create and update assessments, review attempts, manage certificates, and investigate flagged security incidents.

3.4.2 Student Module Design

The student-facing side of the platform is composed of the following major functional units:

- **Authentication and Account Access:** login, registration, refresh tokens, password recovery, and profile hydration.
- **Dashboard Module:** live overview of available assessments, certificates, readiness signals, and recent progress.
- **Assessment Module:** instructions, timed assessment interface, question navigation, flagging, webcam preview, fullscreen monitoring, and final submission.
- **Results Module:** real results history, pagination, summaries, readiness indicators, and detailed result breakdown pages.
- **Practice Mode:** AI-guided session setup, session generation, instant feedback, review reports, weak areas, and study plans.
- **Certificates Module:** certificate listing, detail view, verification sharing, and image download.
- **Career Insights Module:** readiness signals, role-fit suggestions, growth areas, and role spotlight information based on actual student evidence.
- **Settings Module:** profile, notifications, privacy, password change, and device session management.

3.4.3 Administrator Module Design

The administrator-facing system was designed as an operational dashboard. Its major modules include:

- **Admin Overview:** platform metrics, attempts trend, certificate summary, top categories, and activity feed.
- **User Management:** student list, search, filters, pagination, profile inspection, CSV export, account suspension, and access reset.
- **Assessment Management:** assessment creation, editing, publishing, archiving, duplication, and question-bank editing.
- **Attempts Monitoring:** live attempts table, detail pages, grading status, security condition, and export support.
- **Certificates Management:** list, detail, reissue, verify, copy verification ID, and export functionality.
- **Security Alerts:** flagged attempt review, severity display, incident timeline, notes, escalation, review, and flag clearance.
- **Admin Settings:** profile, notifications, privacy, and device-session controls.

3.4.4 Assessment Workflow Design

The assessment subsystem follows a complete lifecycle. First, an administrator creates an exam, defines assessment metadata, and adds or imports questions. When the assessment is published, it becomes visible to students. The student opens the instructions page, accepts the rules, enters fullscreen mode, and begins the timed attempt. During the attempt, answers are stored, question navigation is tracked, and integrity events such as tab switches or webcam issues are recorded. On submission, the backend grades the responses automatically, determines pass or fail status, and issues a certificate if the pass mark is met.

3.5 Database Design

The Prisma schema designed for the project reflects the major entities required by the system. The key entities and their purposes are summarized below.

Entity	Description
User	Stores student and administrator records, including identity, profile, and role information.
UserSession	Stores refresh-token sessions, device identifiers, expiration time, and revocation state.
Exam	Stores each assessment, including title, status, mode, duration, pass mark, and proctoring rules.
Question	Stores question prompt, type, rubric, explanation, difficulty, correct answer, and metadata.
QuestionOption	Stores the answer options for MCQ questions.
ExamAttempt	Stores each student's assessment session, score, percentage, status, device data, and grading summary.
AttemptResponse	Stores submitted answers, awarded marks, and auto-feedback for each question.
ProctorEvent	Stores integrity events such as tab switch, fullscreen exit, multiple-face detection, and suspicious activity.
Certificate	Stores issued certificates, hashes, verification numbers, and links to attempts and exams.
AIArtifact	Stores AI-generated practice sessions, reviews, study plans, and other structured AI outputs.
UserSettings	Stores preferences, notifications, privacy settings, and dashboard presentation choices.

The database design was intentionally normalized so that related records can be tracked independently. For instance, one examination can contain many questions, one student can have many attempts, one attempt can have many proctor events, and one passed attempt can produce one certificate. This structure supports traceability, auditability, and future extensibility.

3.6 Technologies, Tools, and Services Used

Technology / Tool	Purpose in the Project
Next.js (App Router)	Frontend framework used to build the web application interface and page routing.
React 19	Component-based user interface development and client-side interaction handling.
Tailwind CSS	Utility-based styling system for responsive and consistent interface design.
Framer Motion	Subtle animations for page transitions, drawers, cards, and modal interactions.
NestJS	Backend framework for API design, module organization, and business logic.
Prisma ORM	Schema modeling, typed database access, and query abstraction for PostgreSQL.
PostgreSQL	Primary relational database for persistent system storage.
Argon2	Password hashing and authentication security.
JWT (JSON Web Token)	Token-based authentication and role-protected API access.
Gemini API	AI support for practice generation, open-ended grading assistance, and narrative generation.
TensorFlow.js / BlazeFace	Fallback browser-based face detection during proctored assessments.
Native FaceDetector API	Primary browser face-detection path when supported.
SVG-based backend certificate rendering	Generating downloadable certificate files from backend-rendered markup for reliable student downloads.
XLSX	Question import support from spreadsheet files in the assessment authoring process.
Docker Compose	Provisioning PostgreSQL consistently in development.

3.7 Artificial Intelligence Integration Strategy

One of the defining features of the project is the integration of AI into practice, grading, and guidance workflows. The AI integration was not implemented as an isolated chatbot but as a structured support layer around clearly defined educational tasks.

3.7.1 AI in Practice Mode

The practice subsystem uses AI to generate guided practice sessions based on a selected category, topic, difficulty level, and question type. The generated output is normalized into structured practice questions and stored as an AI artifact. When needed, the system falls back to curated topic-aware templates so that the practice mode remains usable even when the AI response is weak or unavailable.

3.7.2 AI in Open-Ended Grading

For multiple choice questions, grading is deterministic and rule-based. However, for theory and certain coding questions, the system applies AI-assisted grading to compare the student response with the expected answer and rubric supplied by the administrator. This allows the platform to assign marks based on semantic closeness, coverage of key ideas, and response quality rather than strict text matching alone. Where AI is unavailable, a fallback heuristic grading approach is used.

3.7.3 AI in Career Insight Generation

Career insight generation is grounded in real student evidence such as scores, open-ended response quality, consistency, integrity discipline, and certification history. AI can optionally refine the narrative explanation, but the underlying role-fit logic is derived from measurable platform data rather than random or purely generic output. This makes the career guidance more credible and explainable.

3.7.4 Gemini Assessment Submission and Grading Data Flow

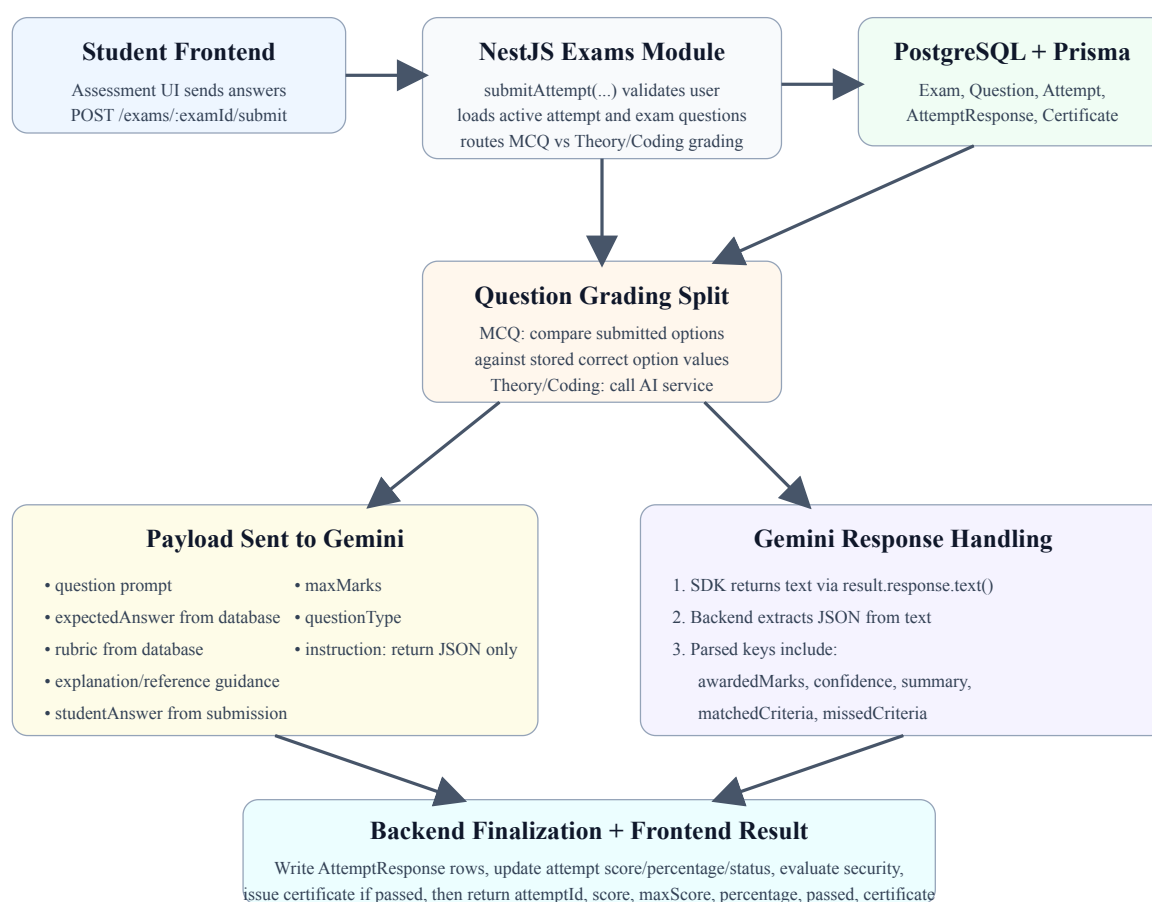
In the current implementation, Gemini AI participates in the system only after a student submits a real assessment and only for **theory** and open-ended **coding** questions. The frontend does not call Gemini directly. Instead, the student interface submits answers to the backend, the backend loads the corresponding assessment records from the database, and the backend decides whether a question should be graded rule-based or sent to Gemini for AI-assisted grading. This design ensures that the grading logic remains centralized, auditable, and protected from direct client-side tampering.

The submission lifecycle begins when the student completes the assessment and the frontend calls `POST /exams/:examId/submit`. The request body contains the list of answers keyed by

question identifiers. At this point, the backend fetches the active attempt, the parent exam, and the full question records, including options, correct answers, rubrics, explanations, and marks. Multiple choice questions are graded immediately by comparing the submitted option values against the stored correct option values. Theory and open-ended coding responses are instead passed into `AIService.gradeOpenEndedResponse(...)`.

The backend does not ask Gemini to guess correctness in isolation. Gemini is supplied with structured grading context assembled from the database, namely the original question prompt, the administrator's expected answer, the rubric, the explanatory reference text, the student's submitted answer, the maximum marks for the question, and the question type. In other words, Gemini is used as a grading assistant that compares the student's response against teacher-defined expectations already stored in the system.

Figure 3.1: Gemini-Assisted Assessment Submission Data Flow



```

flowchart TD
    A[Student Frontend] -->|POST /exams/:examId/submit| B[NestJS Exams Module]
    B --> C[(PostgreSQL via Prisma)]
    B --> D{Question type}
    D -->|MCQ| E[Rule-based grading]
    D -->|Theory / Open-ended Coding| F[AIService.gradeOpenEndedResponse]
    F --> G[Gemini 1.5 Flash]
    G -->|text response containing JSON| F
    F --> H[Backend JSON extraction + validation]
    H --> I[Write AttemptResponse rows]
    I --> J[Update ExamAttempt score, percentage, status, gradingSummary]
    J --> K[Evaluate security review]
    K --> L{Pass threshold met?}
    L -->|Yes| M[Issue certificate]
    L -->|No| N[No certificate]
    M --> O[Return result payload to frontend]
    N --> O

```

Figure 3.1 shows that the backend stands between the student and Gemini at all times. The frontend only knows that the student has submitted answers and later receives the final computed result. The backend performs the protected steps: loading trusted grading context from the database, invoking Gemini where needed, parsing Gemini output, storing marks and feedback, evaluating pass/fail status, and issuing a certificate where the pass threshold is met.

3.7.4.1 Step-by-Step Submission Lifecycle

1. The student completes the assessment and the frontend sends the answers to `POST /exams/:examId/submit`.
2. The backend locates the authenticated student's active in-progress attempt for that exam.
3. The backend loads the exam questions, question options, stored correct answers, rubric, explanation, and marks from the database.
4. The backend loops through all questions and maps each submitted answer by question identifier.
5. For MCQ questions, the backend compares submitted option values against the stored correct option values and awards either full marks or zero.

6. For theory questions, the backend sends the grading context to Gemini through `AIService.gradeOpenEndedResponse(...)`.
7. For coding questions, the backend first checks exact equality against the stored expected answer. If there is no exact match, the backend sends the grading context to Gemini.
8. Gemini returns a text response which the backend parses into JSON using the internal JSON extraction helper.
9. The backend writes the awarded marks and auto feedback for each question into the `AttemptResponse` table.
10. The backend updates the overall attempt status, score, maximum score, percentage, and grading summary in `ExamAttempt`.
11. The backend evaluates the attempt's security state against stored proctoring events.
12. If the student meets the pass threshold, the backend issues a certificate and links it to the attempt.
13. The backend returns the final grading result to the frontend, and the frontend immediately shows the student the score, percentage, outcome, and certificate information if available.

3.7.4.2 Pseudocode View of the Grading Loop

```

receive submitted answers
load active attempt + exam + questions from database
for each question in exam:
    get submitted answer for question
    if question is MCQ:
        compare submitted options with stored correct options
        award marks deterministically
    else if question is THEORY:
        send prompt, expectedAnswer, rubric, explanation, studentAnswer,
maxMarks, questionType to Gemini
        parse JSON-like grading response
    else if question is CODING:
        if submitted answer exactly matches stored expected answer:
            award full marks
        else:
            send grading context to Gemini
            parse JSON-like grading response
    store awarded marks and auto feedback
sum total score
compute percentage

```

```

update attempt status to AUTO_GRADED
evaluate security review
if pass threshold met:
    issue certificate
return score, percentage, pass/fail, certificate

```

3.7.4.3 What the Backend Sends to Gemini

Field Sent	Purpose in Grading
Question prompt	Provides the original question statement Gemini must evaluate.
Expected answer	Supplies the teacher-defined answer stored in the database.
Rubric	Supplies grading criteria, key points, and expected coverage where available.
Reference explanation	Provides extra explanatory context already stored for the question.
Student answer	Provides the actual response being graded.
Maximum marks	Constrains the score Gemini is allowed to award.
Question type	Tells Gemini whether it is grading theory or coding.
Prompt instruction to return JSON only	Guides Gemini toward a machine-readable grading result.

3.7.4.4 What Gemini Returns to the Backend

In the present implementation, Gemini is instructed to return JSON only, but the SDK response is first received as text by the backend through `result.response.text()`. The backend then extracts the JSON object from that text and parses it before using it in grading. Therefore, the effective response handling is **text transport with structured JSON content inside the text**.

Returned Key	Meaning
awardedMarks	The marks Gemini recommends for the response.
confidence	A value between 0 and 1 indicating Gemini's confidence in the grading.
summary	A concise textual explanation of the grading decision.

Returned Key	Meaning
matchedCriteria	Criteria or ideas the student covered successfully.
missedCriteria	Important ideas or criteria the student failed to cover.

3.7.4.5 Gemini Configuration Details

The Gemini integration currently uses the **gemini-1.5-flash** model through the official Google Generative AI SDK in the backend. The Gemini API key is loaded from backend environment configuration through the **GEMINI_API_KEY** setting and then accessed inside the application through the NestJS configuration service. No API secret is exposed to the frontend.

The current code does **not** set an explicit temperature value for Gemini requests. As a result, the application relies on the model SDK's default sampling behaviour. This is an important implementation detail because it means the current integration controls Gemini mainly through prompt design and backend validation rather than through explicit temperature tuning.

3.7.4.6 Fallback Behaviour and Reliability Control

To prevent grading failure from breaking the assessment lifecycle, the AI module includes a fallback heuristic grading method. If Gemini is unavailable, throws an error, or returns text that cannot be parsed into usable JSON, the backend falls back to its own heuristic scoring logic. This fallback uses normalized text, keyword overlap, coverage of reference ideas, and answer length to estimate a fair score. The fallback result is then stored and returned in the same overall attempt-processing pipeline.

3.8 Security and Integrity Design

Because GradAssess AI handles real assessment sessions, security and integrity features were built directly into both the frontend and backend. The purpose is not only to score students but also to preserve confidence in the validity of the results.

3.8.1 Authentication and Access Control

Authentication uses role-aware JWT-based access control. Students and administrators are authenticated through the backend auth module. Refresh tokens are stored in session records so that sessions can be revoked, refreshed, or invalidated after password changes or logout actions. Protected routes are enforced both at the backend controller level and in the frontend application flow.

3.8.2 Assessment Integrity Monitoring

During a real assessment, the following security events are monitored:

- Tab switching out of the exam view.
- Exiting fullscreen mode.
- Webcam permission loss or status change.
- Multiple face detection in the camera feed.
- No face detected for a period of time.
- Suspicious activity determined by accumulated event severity.

These events are logged as *ProctorEvent* records tied to a specific attempt. The backend increments suspicious flags and may escalate the attempt status where necessary. Administrators can later review these records from the Security Alerts dashboard.

3.8.3 Admin Security Review Workflow

The admin security dashboard provides live access to flagged sessions, including their risk level, warning count, event timeline, device information, IP information, notes, and review state. Incidents can be marked as reviewed, escalated, or cleared. This allows the platform to preserve both automation and human oversight.

3.9 Implementation Procedure

The implementation procedure was organized to move from the foundation of the system to the more intelligent and specialized features. The major implementation stages were as follows:

1. Set up the frontend and backend workspaces and establish the project structure.
2. Design the Prisma schema and configure PostgreSQL persistence.
3. Implement user authentication, authorization, and session handling.
4. Build the reusable student and admin dashboard layouts.
5. Create the assessment authoring flow and question management process.
6. Implement live assessment delivery, timing, fullscreen control, webcam handling, and submission flow.
7. Build automatic grading, pass/fail determination, and certificate issuance.
8. Implement practice mode and connect it to AI generation and review endpoints.

9. Develop results analysis, career insights, certificate management, and settings pages.
10. Integrate admin views for users, attempts, certificates, and security incidents.
11. Test, correct, and refine the complete user experience.

3.10 Testing and Validation Approach

Testing was carried out at different levels throughout development. At the technical level, project builds were repeatedly validated to ensure that both frontend and backend compiled successfully after feature integration. At the functional level, user flows were tested manually to confirm that pages, route transitions, API integration, and interface interactions behaved as expected.

Particular attention was given to high-risk workflows such as authentication, assessment start and submission, score generation, certificate issuance, user settings persistence, admin review actions, and security event logging. The project also relied on iterative UI correction, where layout defects, state-refresh issues, and dark/light mode inconsistencies were identified and fixed incrementally.

3.10.1 Areas Specifically Validated

- Authentication and role-based redirection.
- Frontend to backend API integration.
- Assessment timing and submission lifecycle.
- Automatic MCQ grading and AI-assisted open-ended grading.
- Certificate issuance for successful attempts.
- Student dashboard, results, certificates, and practice pages.
- Admin overview, users, assessments, attempts, certificates, and security alerts.
- Persistence of settings, session management, and refresh flows.

3.11 Hardware and Software Requirements

3.11.1 Software Requirements

- Operating System: Windows, Linux, or macOS.
- Node.js runtime.
- PostgreSQL database server.
- Modern web browser such as Google Chrome or Microsoft Edge.

- Docker and Docker Compose for local database provisioning (optional but useful).

3.11.2 Hardware Requirements

- A computer system capable of running a modern web browser.
- Stable internet access for AI-backed features and client-server communication.
- Webcam support for proctored assessments.
- Sufficient memory and storage for running the frontend, backend, and database services locally.

3.12 Summary

This chapter has explained the methodology and technical approach used in building GradAssess AI. The system was developed using an Agile incremental methodology and implemented through a modular client-server architecture. The frontend was built with Next.js, React, Tailwind CSS, and Framer Motion, while the backend was implemented with NestJS, Prisma, and PostgreSQL. AI support was added through Gemini for practice generation, open-ended grading assistance, and narrative guidance, while security and integrity were strengthened through browser-side monitoring and backend proctor-event logging.

The final design supports both student and administrator workflows and provides a complete assessment ecosystem that includes exam delivery, automatic grading, certificate issuance, settings management, results analytics, career insights, and security review. The next chapter can therefore focus on implementation results, screenshots, evaluation, or system testing outcomes, depending on the structure adopted for the remainder of the project report.